

🔧 FSS DATA CODE TOOL · V1.0

SQL Code Generator

Tự động sinh file SQL chuẩn hóa từ YAML Mapping
theo kiến trúc Medallion — tầng Atomic



Python



PyYAML



dbt Macros



Hypothesis

BỐI CẢNH

Vấn đề & Giải pháp

✗ Trước đây

- Viết SQL thủ công cho từng mapping — mất thời gian, dễ sai
- Không nhất quán format, header, alias alignment
- Khó maintain khi có thay đổi YAML schema
- Xử lý CTE phức tạp (merge, chain, unpivot) dễ nhầm logic
- Không có batch processing hoặc error summary

✓ SQL Code Generator

- Parse YAML → sinh SQL hoàn chỉnh, chuẩn hóa hoàn toàn tự động
- Format nhất quán: header, indent, leading comma, alias alignment
- Hỗ trợ dual-format YAML (production & test), auto-detect
- Xử lý Physical / Derived / Unpivot CTE với merge & chain logic
- Batch mode, execution summary, error per file — không dừng batch

KIẾN TRÚC

Medallion Architecture & Processing Pipeline



LUỒNG XỬ LÝ



TÍNH NĂNG

11 Yêu cầu Chức năng



R1 · Parse YAML

Dual-format: Real & Simple. Auto-detect theo YAML root



R2 · Model Name

Derive tên SQL từ YAML: bỏ prefix, replace, lowercase



R3 · Header

Block comment chuẩn hóa: code, name, created, modified



R4 · Physical CTE

dbt ref() macro tự động, WHERE filter từ YAML



R5 · Derived CTE

Merge với physical, chain, aggregation + GROUP BY



R6 · Unpivot CTE

UNPIVOT FOR...IN, mỗi leg một dòng riêng



R7 · Main SELECT

5 transform rules, hash_id → surrogate_key, CAST, NULL



R8 · Filters

WHERE, GROUP BY, HAVING, ORDER BY đúng thứ tự



R9 · Format SQL

Leading comma, indent 4sp, alias alignment, uppercase KW



R10 · Summary

Báo cáo success/fail/time, output paths, error reasons



R11 · CLI & Batch

4 modes: all / single file / glob pattern / folder — continue on error, error collected per file

DEEP DIVE · R1

YAML Parsing — Dual Format



Real Format (production)

- Detect: `mapping` key là dict ở root
- `source_type` : `physical_table` | `derived_cte` | `unpivot_cte`
- `relationship` : join definitions dạng objects
- `final_filter` : list objects có `clause_type` + expression
- `select_fields` : comma-separated string, smart split
- `unpivot_spec` : `value_columns`, `type_column`, `legs`



Simple Format (tests)

- Detect: `mapping_id` ở top-level
- `row_type` : equivalent `source_type`
- `join_definitions` : dạng list đơn giản
- `final_where` / `final_group_by` dạng list strings
- Cùng output: MappingFile dataclass thống nhất
- Dễ viết fixtures, giảm boilerplate khi test

⚠ Validation: thiếu `mapping.id`, `table_name`, `alias`, `target_column` → báo lỗi với row index + field name · Preserve order theo YAML

DEEP DIVE · R4 R5 R6

CTE Generation — 3 Loại

Physical Table CTE

SELECT fields từ YAML · FROM dùng {{
ref('ods_fims_fundcompany')
}}

dbt ref =
namespace_subsystem_table
Subsystem parse từ mapping.id
(segment sau dấu - cuối, trước
.)

WHERE filter nếu field filter
không null

Derived CTE

Merge: derived theo sau
physical, alias khớp → gộp thành
1 CTE: SELECT physical +
aggregation, GROUP BY

Chain: chuỗi derived → mỗi CTE
reference alias CTE trước

Standalone: không match
predecessor → CTE độc lập

Unpivot CTE

Sinh UNPIVOT expression ·
FOR...IN clause

Mỗi leg (column-label pair) trên
1 dòng riêng:

(IdNo, IdDate) AS
'BUSINESS_LICENSE'

Source từ alias physical CTE đứng
trước

```
-- Physical Table CTE (có filter) WITH fu_co AS ( SELECT Id, Name, Capital FROM {{
ref('ods_fims_fundcompany') }} WHERE Status = 'ACTIVE' ), -- Derived CTE chain (fu_co → fu_co_agg)
fu_co_agg AS ( SELECT FundId, COLLECT_LIST(NAMED_STRUCT('bsn_tp_codes', BuId)) AS bsn_tp_codes FROM
fu_co GROUP BY FundId )
```

DEEP DIVE · R7 R8 R9

SQL Generation & Formatting

⚙️ 5 Transformation Rules (theo priority)

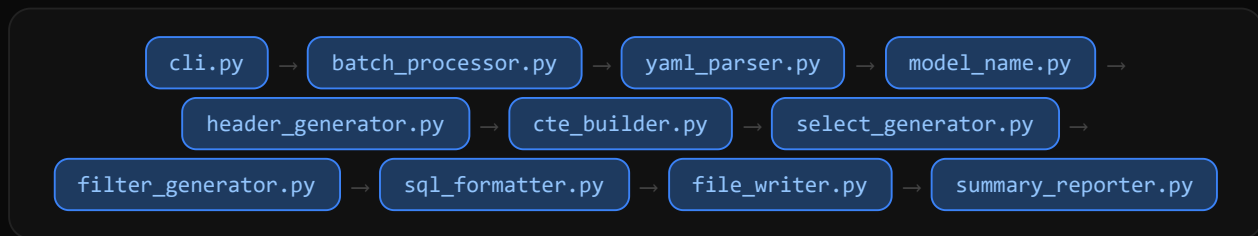
#1	{hash_id}(...)	→ surrogate_key_bigint
#2	transform + data_type	→ CAST(expr AS type)
#3	transform only	→ expr verbatim
#4	data_type only	→ CAST(NULL AS type)
#5	neither	→ NULL
+ Thêm AS target_column ở mọi case		

🌟 Formatting Rules

- **Leading comma** style trong SELECT: dòng đầu không có comma, các dòng sau bắt đầu bằng ,
- **Alias alignment** : pad expression AS thẳng sao cho tất cả cột
- **Indent** 4 spaces: CTE body, JOIN clause
- **WHERE** continuation: indent 8 spaces, AND ở đầu dòng
- **SQL keywords** UPPERCASE: SELECT, FROM, WHERE, JOIN, CAST...
- **COLLECT_LIST/SET** với NAMED_STRUCT → multi-line format

THIẾT KẾ

Module Architecture — Pipeline Pattern



🔧 Core Pipeline Modules

- `yaml_parser` — Dual-format YAML → MappingFile dataclass
- `model_name` — Derive tên file SQL từ YAML filename
- `header_generator` — Program Header SQL comment block
- `cte_builder` — Xây dựng Physical / Derived / Unpivot CTEs
- `select_generator` — Main SELECT với transform rules
- `filter_generator` — WHERE / GROUP BY / HAVING / ORDER BY
- `sql_formatter` — Assemble + align + format toàn bộ SQL

🎭 Orchestration & Data Models

- `cli` — Entry point, 4 dispatch modes
- `batch_processor` — Orchestrate pipeline per file
- `file_writer` — Ensure output dir + write SQL
- `summary_reporter` — Format & print execution summary

DESIGN PRINCIPLES

- Pure functions — dễ test, dễ compose
- Python dataclasses cho intermediate repr
- Chỉ cần PyYAML, no heavy deps

SỬ DỤNG

CLI Interface — 4 Modes

```
$ python -m src
  📁 Process tất cả *.yaml trong Mapping/ recursively → Code/

$ python -m src mapping_atm_fnd_mgt_co-FIMS.FUNDCOMPANY
  📄 Single file – parse layer (atm→Atomic), tìm trong Mapping/Atomic/

$ python -m src "mapping_atm_*-FIMS.*.yaml"
  🔍 Glob pattern – fnmatch search trong Mapping/ recursively

$ python -m src --folder Mapping/Atomic/Involved_Party
  📁 Folder mode – batch process một concept cụ thể

$ python -m src --help
  ? Show usage, options, examples, configured paths
```

📖 Layer Abbreviation Map

atm →	stg →	dtm →
Atomic	Staging	DataMart

📁 Output Path Structure

```
Mapping/Atomic/Involved_Party/file.yaml
→ Code/Atomic/Involved_Party/model.sql
```

CHẤT LƯỢNG

Testing Strategy — 20 Properties

20

Property Tests
Hypothesis

100

Iterations
per property

11

Unit Test
modules

3

Integration
test suites

Unit Tests

test_yaml_parser · test_model_name ·
test_header_generator · test_cte_builder ·
test_select_generator · test_filter_generator ·
test_sql_formatter · test_cli · test_batch_processor ·
test_file_writer · test_summary_reporter

Property-Based Tests (Hypothesis)

P1 YAML round-trip · P2 Model name normalization ·
P3 Output path · P4 Program name · P5 Header
content · P6-P7 Physical CTE + dbt ref · P8-P10
Derived CTE merge/chain/standalone · P11 Unpivot
CTE · P12 Column transform · P13 Filter ordering · P14
Alias alignment · P15 Uppercase KW · P16 SQL
structure · P17-P20 Summary, batch, CLI, format-
detect

Integration Tests

End-to-end: YAML fixture → compare SQL golden file
Batch: mixed valid/invalid files → verify summary
counts
Filesystem: dir creation, file writing, recursive discovery

Test Structure

tests/unit/ · tests/property/ ·
tests/integration/ · tests/fixtures/

Fixtures: valid_mapping.yaml ·
expected_output.sql · invalid_mappings/

OUTPUT DEMO

SQL Output Example

INPUT:
MAPPING_ATM_FND_MGT_CO-
FIMS.FUNDCOMPANY.YAML

```
mapping: id: MAP-ATM-
fnd_mgt_co-
FIMS.FUNDCOMPANY
logical_name: "Mapping
Atomic - Fund Management
Company - source
FIMS.FUNDCOMPANY" input:
- source_type:
physical_table namespace:
ods table_name:
FUNDCOMPANY alias: fu_co
select_fields: "Id, Name,
Capital" filter: "Status
= 'ACTIVE'" relationship:
- join_type: LEFT JOIN
join_alias: fu_bu
join_on: "fu_co.Id =
fu_bu.FundId" -
join_type: LEFT JOIN
join_alias: fu_ty
join_on: "fu_co.Id =
fu_ty.FundId"
mapping_columns: -
target_column:
fnd_mgt_co_id
transformation: "
{hash_id}
('FIMS_FUNDCOMPANY',
fu_co.Id)" -
target_column:
fnd_mgt_co_code
transformation: fu_co.Id
data_type: string -
target_column:
src_stm_code
transformation:
"'FIMS_FUNDCOMPANY'"
data_type: string -
target_column:
charter_cptl_amt
transformation:
fu_co.Capital data_type:
decimal(23,2) -
target_column:
license_dcsn_dt
data_type: date
```

OUTPUT: ATM_FND_MGT_CO_FIMS_FUNDCOMPANY.SQL

```
/*
-----*
* Program code: atm_fnd_mgt_co_fims_fundcompany
* Program name: Atomic Fund Management Company FIMS.FUNDCOMPANY
* Created by:
* Created date:
*-----*
* Modified management
* Date          User          Description
*
*-----*
*/

WITH fu_co AS (
    SELECT Id, Name, Capital
    FROM {{ ref('ods_fims_fundcompany') }}
    WHERE Status = 'ACTIVE'
)

SELECT
    {{ generate_surrogate_key_bigint('FIMS_FUNDCOMPANY', fu_co.Id) }} AS fnd_mgt_co_id
    , CAST(fu_co.Id AS string) AS fnd_mgt_co_code
    , CAST('FIMS_FUNDCOMPANY' AS string) AS src_stm_code
    , CAST(fu_co.Capital AS decimal(23,2)) AS charter_cptl_amt
    , CAST(NULL AS date) AS license_dcsn_dt
FROM fu_co
LEFT JOIN fu_bu ON fu_co.Id = fu_bu.FundId
LEFT JOIN fu_ty ON fu_co.Id = fu_ty.FundId
;
```

RELIABILITY

Error Handling Strategy

⚠ Error Categories

- FileNotFound** YAML path không tồn tại
- YAMLParseError** Invalid YAML syntax + line number
- ValidationError** Missing required field, row index
- AliasError** Empty alias trên physical_table
- UnresolvedRef** derived_cte alias không tồn tại
- UnpivotError** Zero column-label pairs
- PatternMatch** Glob không match file nào

✅ Handling Principles

- **Fail-fast per file** , continue batch — không dừng toàn bộ xử lý
- **No partial output** — hoặc SQL file hoàn chỉnh hoặc không ghi gì
- **Error context đầy đủ** : file path + pipeline step + row/line number + description
- **Validation sớm** trong parse — trước khi sinh SQL
- **CLI errors** — exit ngay với non-zero exit code + message to stderr

```
9 pipeline steps • Mỗi step có error_step label
parsing → model_name → output_path →
header → cte_building →
select_generation → filter_generation
→ formatting → file_writing
```

TỔNG KẾT

SQL Code Generator — Key Takeaways

**Tự động hóa hoàn toàn**

YAML → SQL trong một lệnh.
Batch mode, glob, single file
đều được hỗ trợ



Pipeline modular, pure
functions, dataclasses — dễ
extend cho Staging, DataMart
Kiến trúc mở rộng

**SQL chuẩn hóa**

Header, leading comma, alias
alignment, uppercase
keywords — nhất quán 100%

**Dual-format YAML**

Auto-detect Real Format
(production) và Simple Format
(tests) — cùng output

**20 Property Tests**

Hypothesis PBT bao phủ toàn
bộ correctness properties —
100 iterations mỗi test

**Error Resilient**

Fail-fast per file, no partial
output, full error context với
step và row number

v1.0 — Tầng Atomic (ODS → Atomic) · Next: Staging Layer, DataMart Layer